Project - High Level Design

on

KnowDocs

Datagami GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Sneha Jain	EN22CS301966
2	Soumya Barve	EN22CS301975
3	Sumit Pawar	EN22CS3011000
4	Vikas Devnani	EN22CS3011080
5	Vikas Dhakad	EN22CS3011081

Group Name: D909

Project Number: G-09

Industry Mentor Name: Yash Bansal

University Mentor Name: Vineeta Rathore

Academic Year: 2026

1. Introduction.

1.1. Scope of the document.

This document describes the high-level design of **Know Docs**, a RAG-based chatbot system that lets users query technical documents. Know Docs uses a "Retrieval-Augmented Generation" approach: it augments an LLM (via the Groq API) with relevant information fetched from an internal document knowledge base. The system handles document ingestion (PDF/HTML), semantic indexing (via embeddings and FAISS), and runtime query answering. This HLD covers the architecture, components, data flow, and interfaces of Know Docs.

1.2. Intended Audience

This design is intended for developers, architects, and stakeholders implementing or reviewing the Know Docs system. It assumes familiarity with LLM concepts and basic web service design.

1.3. System overview.

Know Docs consists of a **frontend** (user interface) and **backend services**. Users interact through a web chat UI built with Streamlit. When a query is submitted, the backend retrieves relevant document passages from a vector database (FAISS) and calls an LLM to generate an answer. The backend also handles **document ingestion**: parsing uploaded documents, extracting text chunks, computing embeddings (using a Sentence-Transformer model), and storing them in FAISS for future retrieval. This overall pipeline ensures the LLM's answers are grounded in the actual document content, mitigating "hallucination" by providing up-to-date, task-specific knowledge.

2. System Design.

2.1. Application Design

Users access the Streamlit-based web UI to upload documents and ask questions. The UI forwards document uploads and user queries to backend REST APIs. The backend processes uploads by extracting and chunking document text, embedding each chunk, and storing these vectors in the FAISS index. At query time, the user's question is embedded and used to perform a similarity search in FAISS. The top relevant chunks are retrieved and inserted into a prompt template for the LLM (Groq API) to answer. All components are containerized (e.g. via Docker Compose) for portability.

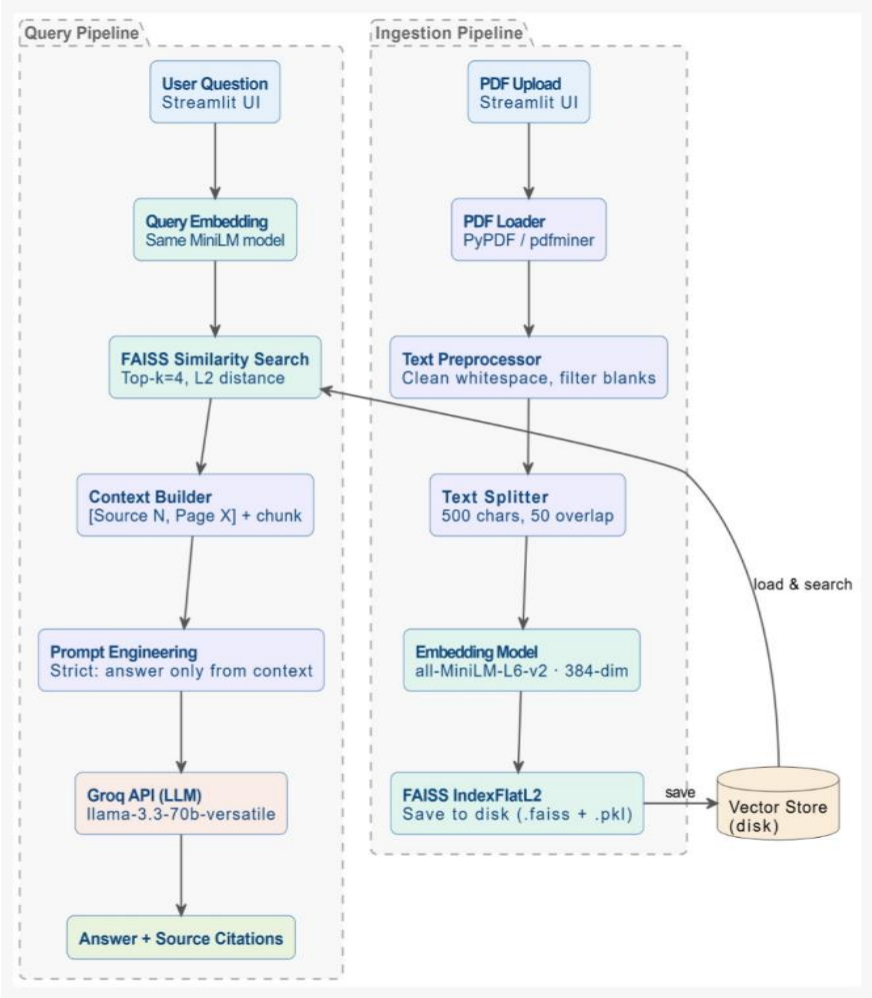


Figure 1: High-level architecture of the Know Docs system.

The architecture has three main layers:
- **Frontend (UI):** A Streamlit application provides a chat interface. It uses Streamlit's chat elements (st.chat_message, st.chat_input) and session state to manage conversations. Users can upload new documents or enter queries.
- **Backend (APIs & Processing):** A set of REST endpoints (e.g. /upload, /query) implemented in Python (e.g. FastAPI) handles business logic. Document uploads trigger the ingestion pipeline; query requests trigger retrieval and LLM generation. The backend also manages the FAISS vector index for retrieval.
- **Data Layer:** A FAISS vector store holds all document chunk embeddings for similarity search. Metadata (e.g. document IDs, chunk texts) is kept in a simple database or file store. An external LLM service (Groq API) provides generation capabilities.

2.2. Process Flow.

The RAG workflow is two-staged: **ingestion** and **inference**.

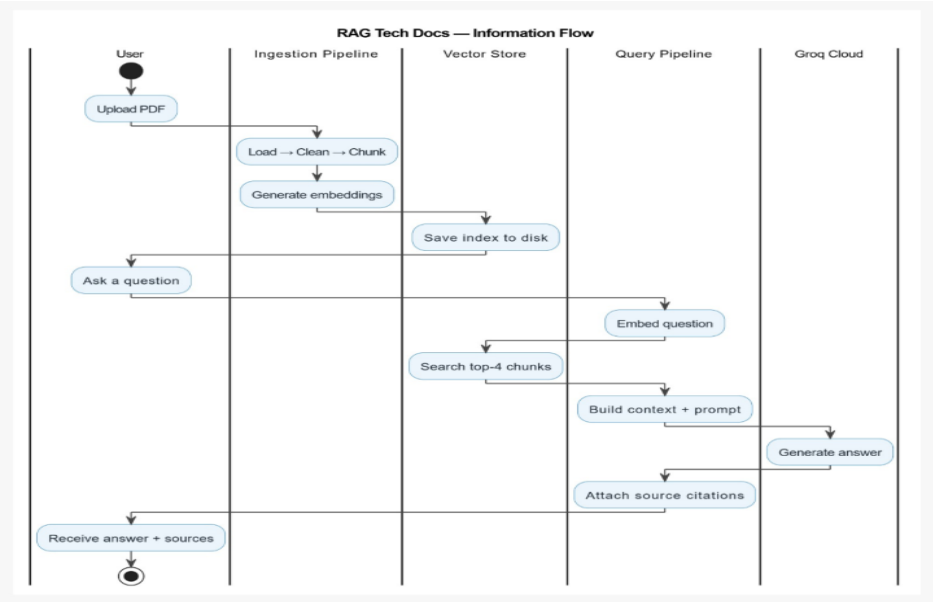
- **Ingestion Stage:** New documents (e.g. technical PDFs) are parsed to extract raw text. The text is then split into overlapping chunks (to preserve context). Each chunk is passed through a sentence-transformer model to compute its embedding vector. These vectors are inserted into FAISS for indexing. During ingestion, "data is transformed… into embeddings" and stored in a vector database for fast retrieval. This builds the reusable knowledge base used at query time.
- **Inference Stage:** When the user asks a question, the backend embeds the query in the same vector space. FAISS performs a **nearest-neighbor search** to find the most similar document chunks. The retrieved chunks form the "context" for the LLM. This context is inserted into a prompt template such as:
  Context:
  <relevant document excerpts>
  Question: <user query>
  Answer:

  The LLM (Groq) then generates the answer, combining its learned knowledge with the provided context Because the answer is grounded in retrieved facts, accuracy is improved (reducing hallucination).

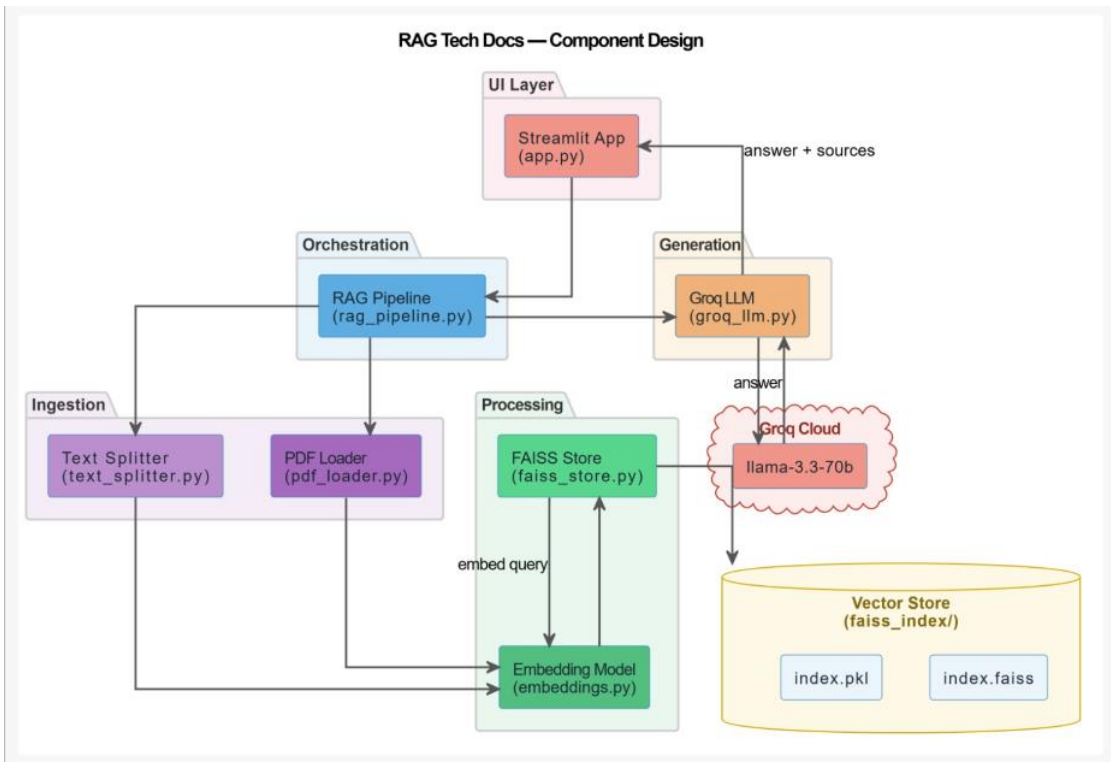
In summary, the process flow is: (1) **Embed Documents** → **Index** during ingestion; (2) **Embed Query** → **Retrieve** → **Augment Prompt** → **Generate** during inference. Figure 2 (below) illustrates this pipeline.

2.3. Information Flow.

The information flow closely follows the process flow. The user's query flows from the UI to the backend, then into FAISS, and finally into the LLM. The LLM's answer flows back through the backend to the UI. The document text flows from the original files into embeddings and into FAISS, then as needed back out into prompts. All data transfers use secure channels. Internal data transformations (e.g. text → embedding) occur within the backend. The flow is symmetric for question and answer, ensuring context is consistently integrated.



2.4. Components Design



- **Document Ingestion Pipeline:** Handles file upload and parsing. We use Python libraries to extract text from PDFs. The text is preprocessed (e.g. cleaning, removing tables or irrelevant references) to improve quality. It is then split into chunks of a few hundred words each (possibly overlapping). Effective chunking is key: retrieval quality depends on how content is segmented.

- **Embedding Engine:** Each text chunk is fed into a pretrained Sentence-Transformer model (e.g. `all-MiniLM-L6-v2`). This produces a fixed-size vector (e.g. 384 dims) representing the chunk's semantics. Embedding inference is batched for efficiency. If the embedding model is ever updated, all documents must be reprocessed (older embeddings become incompatible).
- **Vector Store (FAISS):** We use FAISS to index and search embeddings. FAISS was chosen for its high speed and memory-optimized design. We typically use an IndexFlatL2 (exact search) or an IVF-PQ index for scale. FAISS indexes live in RAM for fast lookup. Metadata for each chunk (document ID, chunk text) is stored alongside or in a parallel store. We persist the FAISS index to disk after building so it can be quickly reloaded on restart.
- **Query Processor:** On each query, the backend obtains the embedding and queries FAISS to retrieve top-K similar chunks. These chunks are ranked by distance. The processor then composes the prompt. Optionally, we could cache frequent query embeddings or results to speed up repeats.
- **Prompt Template & LLM API:** A template is used to combine retrieved context with the user's query. The Groq API (OpenAI-compatible) is called with this prompt. API keys and endpoints are secured. The LLM returns a text answer, which is sent back to the frontend.
- **Frontend UI:** Built with Streamlit, the UI provides chat message containers and input boxes. It maintains chat history using session state so the conversation persists. On the upload page, users can select files; on the chat page, users type queries. All UI actions trigger backend API calls over HTTPS.

3. Data Design.

3.1. Data Model

The primary data entities are **Documents**, **Chunks**, and **Embeddings**:
- A **Document** has an ID, title, and metadata (source, date). Each document generates many **Chunks**.
- A **Chunk** has an ID, a parent Document ID, and the chunk text. Chunks are typically fixed-size text segments (e.g. 500 tokens).
- Each Chunk has an **Embedding**: a vector produced by the embedding model. These vectors are stored in FAISS (with a reference back to the chunk ID).
- The LLM generates **Answer** records, optionally stored with the query and response.

This simple model supports linking an answer back to the source chunks via the chunk IDs.

3.2. Data Access Mechanism

- **Write (Ingestion):** Documents are written via the upload API. The backend

  service parses the file and writes chunk data to the metadata store (database or file store) and embeddings to FAISS. The write operations are atomic per-chunk.

- **Read (Query):** The query API reads the FAISS index (in-memory) to find top-K chunks. It then reads the corresponding chunk texts from the metadata store. These are passed to the LLM API, which also reads the prompt and writes an answer. Answers are returned to the user.
- **Index Persistence:** We use FAISS's built-in write_index to save the index to disk after builds, and read_index to load it on startup[13]. This avoids re-indexing on every service restart.
- **Backup and Migration:** Before making schema or model changes, we back up the FAISS index. If the embedding model changes, the entire index must be rebuilt (as older vectors will be invalid)[8].

3.3. Data Retention Policies

All ingested documents and their embeddings are retained indefinitely by default, as the system's knowledge base. Administrators may occasionally purge outdated or irrelevant documents. User queries and answers are logged for auditing but could be purged after a retention period per policy. Embeddings for deleted documents are removed from FAISS to free space.

4. Interfaces

User Interface: A web-based chat application (Streamlit). Users type queries into a chat input and see responses in a chat window. The UI also includes a file upload form. Interaction is via asynchronous API calls (AJAX). The design ensures only authenticated users (if applicable) can use the service.

Backend API: REST endpoints for uploading documents (`POST /upload`), querying (`POST /query`), and status checks (`GET /health`). These use JSON for data exchange and JWT or OAuth2 tokens for authentication.

External Services: The backend calls the LLM API (Groq) over HTTPS, passing in the prompt and receiving the answer. API credentials are secured on the server (not exposed to clients).

Optional CLI/API for Developers: We may expose an internal API or CLI for data import/export or monitoring. This uses the same endpoints.

Each interface is protected by TLS to ensure data privacy and requires proper authentication and authorization.

5. State and Session Management

- **API Security:** All API calls use HTTPS (TLS). The upload/query endpoints require authenticated users. We plan to use OAuth2/OpenID Connect for user auth. The LLM API key is kept secret on the backend and not exposed to clients.
- **Input Validation:** Document uploads and queries are validated/sanitized to prevent injection attacks.
- **Data Privacy:** Sensitive data (if any) is encrypted at rest. Logs contain no raw user data.

- **Scalability:** The system is designed to handle growing document sets and user load. The FAISS index can scale by adding shards or GPU acceleration. The stateless backend can be replicated behind a load balancer.
- **Performance:** Aim for low latency. Retrieval happens in-memory (FAISS) and is very fast. The main delay is the LLM call, so we optimize prompts for brevity. Caching recent query results can save repeated LLM calls.
- **Availability:** The services (frontend, backend, vector store) are monitored and containerized for quick restarts. Critical data (FAISS index, metadata) is backed up.
- **Maintainability:** The design follows modular best practices (separate ingestion vs query logic), making the codebase easier to maintain. Configuration (e.g. model names) is externalized.

6. Caching

The system uses an in-memory cache for recent query embeddings or answers to avoid repeated LLM and FAISS calls. Caching queries and responses can significantly reduce API usage and improve response time.

7. Non Functional Requirements

7.1 **Security Aspects:** All communication is over HTTPS (TLS) to encrypt data in transit. Sensitive endpoints require authentication (e.g. OAuth2 tokens). The LLM API key is stored securely on the server.

7.2 **Performance Aspects:**

The architecture is optimized for low latency and high throughput. The vector search is in-memory, and prompts are concise. Techniques like request batching and parallel processing help maintain performance under load. Metrics (latency, throughput) are monitored to ensure the system meets response-time requirements.

8. References

- *Introduction to LLM RAG – Retrieval Augmented Generation Explained*, Weaviate (Oct 2024).
- *Retrieval Augmented Generation with Groq API*, Groq Blog (Jan 2024).
- *Faiss: A library for efficient similarity search*, Meta Engineering (2017).
- *Mastering Faiss Vector Database: A Beginner's Handbook*, PingCAP (2023.
- *RAG with Streamlit, FastAPI, Weaviate, and Hamilton!*, DAGWorks (Sep 2023).

- *Challenges of Scaling RAG Applications*, MyScale Blog (Jan 2024).
- *Best practices for REST API security*, StackOverflow Blog (Oct 2021).
-
- *Build a basic LLM chat app*, Streamlit Docs.

---

[1] Build a basic LLM chat app - Streamlit Docs

https://docs.streamlit.io/develop/tutorials/chat-and-llm-apps/build-conversational-apps
[2] [3] [6] [7] **Introduction to LLM RAG - Retrieval Augmented Generation Explained | Weaviate**
https://weaviate.io/blog/introduction-to-rag
[4] **Retrieval Augmented Generation with Groq API | Groq is fast, low cost inference.**
https://groq.com/blog/retrieval-augmented-generation-with-groq-api
[5] [8] [16] [18] **Retrieval augmented generation (RAG) with Streamlit, FastAPI, Weaviate, and Hamilton!**
https://blog.dagworks.io/p/retrieval-augmented-generation-reference-arch
[9] [11] **Faiss: A library for efficient similarity search - Engineering at Meta**
https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/
[10] [17] **Mastering Faiss Vector Database: A Beginner's Handbook**
https://www.pingcap.com/article/mastering-faiss-vector-database-a-beginners-handbook/

[12] [19] **Challenges of Scaling Retrieval-Augmented Generation Applications | Medium**
https://medium.com/@myscale/challenges-of-scaling-retrieval-augmented-generation-applications-25fe4abc0f3e
[13] **How do I persist FAISS indexes? - Stack Overflow**
https://stackoverflow.com/questions/78734751/how-do-i-persist-faiss-indexes
[14] [15] **Best practices for REST API security: Authentication and authorization - Stack Overflow**
https://stackoverflow.blog/2021/10/06/best-practices-for-authentication-and-authorization-for-rest-apis/